1. Introduction

1.1 Purpose of the Mathematical Framework

The **Scientific AI Framework** combines natural language processing, machine learning, numerical simulation, and Bayesian inference to provide an end-to-end system for scientific computation. While the software architecture is modular and implementation-driven, its behaviour is fundamentally governed by well-defined mathematical models.

The purpose of this document is to present the mathematical foundations underlying the framework, including:

- The physical and statistical models used for forward simulation.
- The probabilistic formulations employed for parameter estimation and inference.
- The optimization and machine learning techniques used for intent detection and parameter prediction.
- The assumptions and approximations adopted throughout the system.
- The correspondence between mathematical expressions and their software implementation.

Rather than treating the framework as a black-box artificial intelligence system, the design emphasizes interpretable scientific computation in which every prediction or simulation can be traced back to explicit mathematical equations and numerical algorithms.

1.2 Relationship Between Mathematical Models and Software Architecture

The framework follows a layered architecture in which mathematical models are separated from natural language understanding and execution logic.

1. **Natural Language Processing Layer** User queries expressed in natural language are processed by neural intent detection and slot prediction models. These models identify the requested scientific task and estimate the numerical parameters required for execution.

2. **Intent Representation Layer** The extracted information is converted into a structured `ScientificIntent` object that provides a domain-independent representation of the requested computation.

3. **Execution Layer** Domain-specific executors receive the structured intent and invoke the corresponding mathematical models. Examples include hydrogen spectrum simulation, quantum state evolution, or Bayesian parameter inference.

4. **Scientific Computation Layer** Numerical algorithms implement the underlying mathematical equations governing each scientific domain. These computations generate simulated data, evaluate likelihood functions, or estimate unknown parameters.

5. **Inference and Output Layer** Results are returned to the user in the form of simulated observables, inferred parameters, probability distributions, or visualizations.

This separation of concerns ensures that the machine learning components are responsible for interpreting user requests, while the scientific results themselves are produced by deterministic mathematical models or statistically principled inference algorithms.

1.3 Mathematical Philosophy

The framework adopts a hybrid approach that combines data-driven learning with first-principles scientific modelling.

- Neural networks are used to interpret natural language and predict structured intents.

- Domain-specific executors perform computations based on established physical laws and mathematical equations.
- Bayesian inference techniques quantify uncertainty in estimated parameters.
- Monte Carlo methods are employed when analytical solutions to posterior distributions are impractical.
- Numerical linear algebra and optimization methods provide efficient implementations of the underlying models.

Consequently, artificial intelligence acts as an interface for scientific computation rather than replacing the mathematical models themselves.

1.4 General Mathematical Formulation

At a high level, every computational task within the framework can be expressed as a mapping

$$\mathbf{y} = f(\boldsymbol{\theta}),$$

where

- $\boldsymbol{\theta}$ denotes the set of model parameters,
- f represents the domain-specific mathematical model, and
- $\mathbf{y}$ denotes the predicted observable quantities.

For inverse problems, the objective is to estimate the unknown parameters from observed data,

$$\boldsymbol{\theta} \leftarrow \mathbf{y}_{\text{obs}},$$

which is accomplished using Bayesian inference and Markov Chain Monte Carlo sampling.

1.5 Assumptions

Unless otherwise stated, the framework operates under the following general assumptions:

- Mathematical models accurately represent the underlying physical processes within their intended domain of applicability.
- Numerical computations are performed using finite-precision floating-point arithmetic.
- Observational uncertainties are treated as stochastic variables and are commonly modelled using Gaussian statistics.
- Forward models are deterministic for a fixed set of input parameters.
- Bayesian inference assumes that prior distributions and likelihood functions are appropriately specified.
- Markov Chain Monte Carlo algorithms generate approximate samples from posterior distributions after sufficient convergence and mixing.

Individual scientific modules may introduce additional assumptions that are documented in their respective sections.

1.6 Notation

Throughout this document, the following notation is used consistently.

Symbol	Meaning
$\boldsymbol{\theta}$	Vector of model parameters
$f(\cdot)$	Forward model or simulation function
$\mathbf{y}$	Model prediction or simulated observable

Symbol	Meaning
$\mathbf{y}_{\text{obs}}$	Observed or measured data
$P(\cdot)$	Probability distribution
$L(\cdot)$	Likelihood function
H	Hamiltonian operator in quantum mechanical models
$\langle \psi \rangle$	Quantum state vector
t	Time variable
E	Energy variable
σ	Standard deviation or instrumental broadening parameter
B	Constant background contribution
A_i	Amplitude associated with the (i)-th spectral component

Additional symbols specific to individual domains are introduced when first used.

2. Probability and Bayesian Inference

2.1 Overview

Many scientific problems involve estimating unknown parameters from observed data. Such problems are known as **inverse problems**, where the objective is to infer the model parameters that most likely produced the measurements.

The Scientific AI Framework adopts a **Bayesian approach** to parameter estimation. Rather than producing a single deterministic estimate, Bayesian inference represents uncertainty explicitly by computing a probability distribution over the unknown parameters conditioned on the observed data.

This probabilistic formulation enables robust parameter estimation, uncertainty quantification, and principled incorporation of prior knowledge.

2.2 Bayes' Theorem

Bayesian inference is founded on **Bayes' theorem**, which relates the conditional probability of model parameters given observed data to the likelihood of the data under those parameters.

$$P(\boldsymbol{\theta} \mid D) = \frac{P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta})}{P(D)}$$

where

- D denotes the observed data,
- $\boldsymbol{\theta}$ represents the vector of unknown model parameters,
- $P(\boldsymbol{\theta})$ is the **prior distribution**,
- $P(D \mid \boldsymbol{\theta})$ is the **likelihood function**,
- $P(\boldsymbol{\theta} \mid D)$ is the **posterior distribution**, and
- $P(D)$ is the **evidence** or normalization constant.

Since the evidence does not depend on the unknown parameters, Bayesian parameter estimation is commonly expressed as

$$P(\boldsymbol{\theta} \mid D) \propto P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta}).$$

Thus, the posterior distribution is proportional to the product of the likelihood and the prior.

2.3 Prior Distributions

The prior distribution,

$$P(\boldsymbol{\theta}),$$

encodes knowledge or assumptions about the parameters before observing the experimental data.

Depending on the application, priors may be:

- **Uniform priors**, assigning equal probability over a specified interval.
- **Gaussian priors**, expressing prior estimates with associated uncertainty.
- **Domain-specific priors**, derived from theoretical constraints or previous experiments.

Within the Scientific AI Framework, priors serve two primary purposes:

1. Restricting parameter exploration to physically meaningful regions.
2. Incorporating existing scientific knowledge into the inference process.

When no strong prior knowledge is available, broad or weakly informative priors may be employed to allow the observed data to dominate the inference.

2.4 Likelihood Functions

The likelihood function measures how well a proposed parameter vector explains the observed data.

For observed data

$$D = y_1, y_2, \ldots, y_n,$$

and corresponding model predictions

$$f(\boldsymbol{\theta}) = \{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n\}$$

the likelihood is

$$P(D \mid \boldsymbol{\theta}).$$

Gaussian Noise Model

A common assumption in scientific measurements is that observational errors are independent and normally distributed,

$$y_i = \hat{y}_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Under this assumption, the likelihood becomes

$$P(D \mid \boldsymbol{\theta}) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right).$$

For numerical stability, implementations often maximize or evaluate the **log-likelihood** instead,

$$\log P(D \mid \boldsymbol{\theta}) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

The second term shows that maximizing the Gaussian likelihood is equivalent to minimizing the weighted sum of squared residuals.

2.5 Posterior Distribution

Combining the prior and likelihood yields the posterior distribution,

$$P(\boldsymbol{\theta} \mid D) \propto P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta}).$$

The posterior represents the updated belief about the parameters after incorporating the observed data.

Unlike deterministic optimization methods, the posterior provides a complete probabilistic description of parameter uncertainty, including:

- regions of high probability,
- parameter correlations,
- confidence intervals,
- multimodal behaviour, and
- predictive uncertainty.

For many practical scientific models, the posterior cannot be evaluated analytically and must instead be explored numerically using sampling algorithms such as Markov Chain Monte Carlo (MCMC).

2.6 Maximum a Posteriori (MAP) Estimation

One possible summary of the posterior is its mode, known as the **Maximum a Posteriori (MAP)** estimate.

The MAP estimator is defined as

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} P(\boldsymbol{\theta} \mid D).$$

Using Bayes' theorem,

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta}) \right].$$

Taking logarithms,

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\log P(D \mid \boldsymbol{\theta}) + \log P(\boldsymbol{\theta}) \right].$$

MAP estimation produces a single best parameter vector while incorporating prior information.

If the prior is uniform over the feasible parameter space, MAP estimation reduces to **Maximum Likelihood Estimation (MLE)**.

2.7 Full Bayesian Inference

Although MAP estimation provides a convenient point estimate, it discards most of the information contained in the posterior distribution.

Full Bayesian inference instead seeks to characterize the entire posterior,

$$P(\boldsymbol{\theta} \mid D),$$

by generating representative samples from it.

These samples can be used to compute posterior means,

$$\mathbb{E}[\boldsymbol{\theta}] = \int \boldsymbol{\theta} P(\boldsymbol{\theta} \mid D) d\boldsymbol{\theta},$$

posterior variances,

$$\mathrm{Var}(\boldsymbol{\theta}) = \int (\boldsymbol{\theta} - \mathbb{E}[\boldsymbol{\theta}])^2 P(\boldsymbol{\theta} \mid D) d\boldsymbol{\theta},$$

credible intervals, parameter correlations, and predictive distributions.

The Scientific AI Framework adopts this full Bayesian perspective for parameter inference modules, employing Markov Chain Monte Carlo techniques to approximate posterior distributions when closed-form analytical solutions are unavailable.

2.8 Application Within the Framework

For inference tasks implemented in the framework, the computational workflow follows these steps:

1. A forward model predicts observable quantities from a candidate parameter vector $\boldsymbol{\theta}$.
2. The likelihood function evaluates how well the prediction matches the observed data.
3. Prior distributions encode any existing knowledge or physical constraints on the parameters.
4. Bayes' theorem combines the prior and likelihood to define the posterior distribution.
5. Markov Chain Monte Carlo sampling is used to approximate the posterior when direct analytical evaluation is infeasible.
6. Posterior samples are summarized to estimate parameter values and quantify their associated uncertainties.

This probabilistic methodology provides a mathematically rigorous foundation for scientific parameter estimation while explicitly accounting for measurement noise and model uncertainty.

3. Markov Chain Monte Carlo (MCMC)

3.1 Overview

In many scientific inference problems, the posterior distribution

$$P(\boldsymbol{\theta} \mid D)$$

cannot be evaluated analytically or integrated in closed form due to the complexity and high dimensionality of the parameter space. Instead of attempting to compute the posterior directly, one may generate a collection of representative samples whose distribution approximates the posterior.

Markov Chain Monte Carlo (MCMC) is a family of stochastic algorithms designed for this purpose. By constructing a Markov chain whose stationary distribution is the desired posterior, MCMC enables estimation of parameter values, uncertainties, correlations, and other statistical quantities through sampling rather than direct integration.

Within the Scientific AI Framework, MCMC serves as the principal method for Bayesian parameter inference in scientific models such as hydrogen spectroscopy and quantum-system identification.

3.2 Sampling Objective

The objective of MCMC is to generate samples

$$\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \ldots, \boldsymbol{\theta}^{(N)}$$

such that, after convergence,

$$\boldsymbol{\theta}^{(k)} \sim P(\boldsymbol{\theta} \mid D).$$

Once such samples have been obtained, expectations with respect to the posterior distribution can be approximated by sample averages.

For any function $g(\boldsymbol{\theta})$,

$$\mathbb{E}[g(\boldsymbol{\theta})] = \int g(\boldsymbol{\theta}) P(\boldsymbol{\theta} \mid D) d\boldsymbol{\theta} \approx \frac{1}{N} \sum_{k=1}^{N} g!\left(\boldsymbol{\theta}^{(k)}\right).$$

Thus, difficult multidimensional integrals are replaced by averages over sampled parameter vectors.

3.3 Markov Chains

A Markov chain is a stochastic process in which the next state depends only on the current state and not on the complete history of previous states.

If

$$\boldsymbol{\theta}^{(0)}, \boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(N)}$$

denote successive parameter vectors, the Markov property is

$$P!\left(\boldsymbol{\theta}^{(k+1)} \mid \boldsymbol{\theta}^{(k)}, \ldots, \boldsymbol{\theta}^{(0)}\right) = P!\left(\boldsymbol{\theta}^{(k+1)} \mid \boldsymbol{\theta}^{(k)}\right).$$

Appropriate transition rules ensure that the stationary distribution of this chain is the target posterior distribution.

3.4 Ensemble Sampler Principles

The Scientific AI Framework employs an **ensemble sampling strategy**, in which multiple interacting Markov chains, called **walkers**, explore the parameter space simultaneously.

Suppose there are M walkers,

$$\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots, \boldsymbol{\theta}_M.$$

Instead of proposing new states independently using fixed proposal distributions, each walker generates proposals based on the positions of other walkers within the ensemble. This adaptive strategy naturally accounts for parameter scaling and correlations without extensive manual tuning.

The principal advantages of ensemble sampling include:

- improved exploration of correlated parameter spaces,
- reduced sensitivity to proposal scaling,
- efficient performance in moderately high-dimensional problems,
- simultaneous generation of multiple chains for convergence assessment.

Because walkers cooperate during exploration, ensemble samplers often achieve faster mixing than traditional single-chain methods for many scientific applications.

3.5 Acceptance Probability

Each proposed move is either accepted or rejected according to an acceptance criterion that guarantees convergence toward the target posterior distribution.

Let

- $\boldsymbol{\theta}$ denote the current state,
- $\boldsymbol{\theta}'$ denote a proposed state.

The acceptance probability is generally expressed as

$$\alpha = \min\left(1, \frac{P(\boldsymbol{\theta}' \mid D) q(\boldsymbol{\theta} \mid \boldsymbol{\theta}')}{P(\boldsymbol{\theta} \mid D) q(\boldsymbol{\theta}' \mid \boldsymbol{\theta})}\right),$$

where $q(\cdot)$ denotes the proposal distribution.

When symmetric proposals are used,

$$q(\boldsymbol{\theta}' \mid \boldsymbol{\theta}) = q(\boldsymbol{\theta} \mid \boldsymbol{\theta}'),$$

this simplifies to

$$\alpha = \min\left(1, \frac{P(\boldsymbol{\theta}' \mid D)}{P(\boldsymbol{\theta} \mid D)}\right).$$

In practice, implementations commonly evaluate logarithms to improve numerical stability,

$$\log P(\boldsymbol{\theta} \mid D) = \log P(D \mid \boldsymbol{\theta}) + \log P(\boldsymbol{\theta}).$$

A proposal that increases the posterior probability is always accepted, while proposals that decrease it may still be accepted with a probability determined by the ratio above, allowing the sampler to escape local maxima and explore the full posterior distribution.

3.6 Burn-in

The initial states of the walkers are typically not representative of the target posterior distribution.

The early phase of sampling, during which the chains migrate toward regions of high posterior probability, is known as the **burn-in period**.

Samples collected during burn-in are generally discarded because they depend strongly on the initial conditions rather than the equilibrium distribution.

If the total number of generated samples is N and the burn-in length is B, only

$$N_{\text{usable}} = N - B$$

samples are retained for subsequent statistical analysis.

3.7 Convergence and Mixing

Reliable Bayesian inference requires that the Markov chains adequately explore the posterior distribution.

Important indicators of satisfactory convergence include:

- stabilization of sampled parameter values,

- absence of systematic trends in trace plots,
- effective exploration by all walkers,
- low autocorrelation between successive samples,
- agreement among independently initialized chains.

Poor convergence may result from insufficient sampling, inappropriate priors, highly multimodal posteriors, or inadequate initialization.

Increasing the number of walkers or extending the sampling duration often improves convergence and produces more reliable posterior estimates.

3.8 Posterior Summaries

Once convergence has been achieved and burn-in samples have been removed, the retained samples provide an empirical approximation to the posterior distribution.

Given posterior samples

$$\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(N)},$$

common summary statistics include the posterior mean,

$$\bar{\boldsymbol{\theta}} = \frac{1}{N} \sum_{k=1}^{N} \boldsymbol{\theta}^{(k)},$$

and the posterior covariance matrix,

$$\mathrm{Cov}(\boldsymbol{\theta}) = \frac{1}{N-1} \sum_{k=1}^{N} \left(\boldsymbol{\theta}^{(k)} - \bar{\boldsymbol{\theta}} \right) \left(\boldsymbol{\theta}^{(k)} - \bar{\boldsymbol{\theta}} \right)^{T}.$$

In addition, one may compute:

- posterior standard deviations,
- credible intervals,
- marginal probability distributions,
- parameter correlations,
- highest posterior density regions,
- predictive distributions for future observations.

These summaries provide substantially richer information than a single point estimate and allow uncertainties to be propagated through subsequent scientific analyses.

3.9 Application Within the Framework

For inference tasks in the Scientific AI Framework, MCMC is used to estimate unknown model parameters from observed data.

The computational procedure is as follows:

1. Initialize an ensemble of walkers with candidate parameter vectors.
2. Evaluate the forward model for each proposed parameter set.
3. Compute the corresponding log-likelihood and log-prior.
4. Form the log-posterior according to Bayes' theorem.
5. Iteratively update walker positions using the ensemble sampling algorithm.

6. Discard burn-in samples and retain the converged chains.
7. Compute posterior summaries, including parameter estimates and uncertainty intervals, from the retained samples.

This methodology enables statistically rigorous parameter estimation while naturally incorporating prior knowledge, observational uncertainty, and parameter correlations into the inference process.

The current implementation employs the affine-invariant ensemble sampler introduced by Goodman & Weare and implemented in the emcee library. This algorithm is particularly effective for correlated parameter spaces and requires relatively little manual tuning compared with conventional Metropolis–Hastings methods.

4. Forward Modelling

4.1 Overview

Forward modelling is the process of computing observable quantities from a given set of model parameters. In contrast to inverse problems, where unknown parameters are estimated from data, a forward model assumes that the parameters are known and predicts the corresponding physical or experimental observations.

Within the Scientific AI Framework, forward models form the computational core of every scientific domain. They are used both for direct simulation and as components of Bayesian inference, where repeated evaluations of the forward model are required to compute likelihood functions.

Examples of forward modelling within the framework include:

- Simulation of hydrogen emission spectra from transition energies and amplitudes.
- Time evolution of single-qubit quantum systems.
- Dynamics of coupled multi-qubit systems under specified Hamiltonians.
- Generation of synthetic datasets for machine learning and inference validation.

4.2 General Mathematical Formulation

A forward model may be represented abstractly as a mapping

$$\mathbf{y} = f(\boldsymbol{\theta}),$$

where

- $\boldsymbol{\theta}$ denotes the vector of model parameters,
- $f(\cdot)$ represents the mathematical model or simulation procedure,
- $\mathbf{y}$ denotes the predicted observable quantities.

The parameter vector may contain physical constants, amplitudes, frequencies, coupling coefficients, background levels, instrumental parameters, or any other quantities required by the underlying scientific model.

For example,

$$\boldsymbol{\theta} = (\theta_1, \theta_2, \ldots, \theta_p),$$

where p is the number of free parameters.

The output may likewise be represented as

$$\mathbf{y} = (y_1, y_2, \ldots, y_n),$$

where each component corresponds to a predicted measurement or simulated observable.

4.3 Deterministic Nature of the Forward Model

In the absence of measurement noise or stochastic effects, the forward model is assumed to be deterministic.

That is, for a fixed parameter vector,

$$\boldsymbol{\theta},$$

the model always produces the same prediction,

$$f(\boldsymbol{\theta}) = \mathbf{y}.$$

Repeated evaluations with identical inputs therefore yield identical outputs.

This deterministic property is particularly important for Bayesian inference, where the forward model may be evaluated thousands or millions of times during posterior sampling.

4.4 Forward Models in Bayesian Inference

In parameter estimation problems, observed data

$$\mathbf{y}_{\text{obs}}$$

are compared against forward model predictions

$$\mathbf{y}_{\text{pred}} = f(\boldsymbol{\theta}).$$

The discrepancy between prediction and observation determines the likelihood function used in Bayes' theorem.

Defining the residual vector as

$$\mathbf{r} = \mathbf{y}_{\text{obs}} - \mathbf{y}_{\text{pred}},$$

the likelihood is evaluated according to an assumed statistical error model, commonly Gaussian.

Consequently, the forward model acts as the mathematical link between unknown parameters and measurable experimental quantities.

4.5 Noise Assumptions

Although the forward model itself is deterministic, real experimental observations generally contain measurement uncertainty.

The observed data are therefore represented as

$$\mathbf{y}_{\text{obs}} = f(\boldsymbol{\theta}) + \boldsymbol{\varepsilon},$$

where

$$\boldsymbol{\varepsilon} = (\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_n)$$

denotes the measurement noise.

Gaussian Noise Model

Throughout much of the framework, the observational noise is assumed to follow an independent Gaussian distribution,

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where

- the expected value is

$$\mathbb{E}[\varepsilon_i] = 0,$$

- and the variance is

$$\mathrm{Var}(\varepsilon_i) = \sigma^2.$$

Under this assumption,

$$y_{\mathrm{obs},i} = f_i(\boldsymbol{\theta}) + \varepsilon_i.$$

The Gaussian model is widely adopted because it provides a mathematically convenient approximation for many experimental measurement processes and leads naturally to least-squares optimization and Gaussian likelihood functions.

Independence Assumption

Unless otherwise specified, the framework assumes that individual measurement errors are statistically independent,

$$\mathrm{Cov}(\varepsilon_i, \varepsilon_j) = 0, \qquad i \neq j.$$

This assumption simplifies likelihood evaluation and is appropriate for many physical measurement systems in which observations are acquired independently.

Future extensions may incorporate correlated or heteroscedastic noise models where scientifically justified.

4.6 Synthetic Data Generation

An important application of forward modelling within the Scientific AI Framework is the generation of synthetic datasets.

Synthetic observations are produced by evaluating the forward model for known parameters,

$$\mathbf{y}_{\mathrm{true}} = f(\boldsymbol{\theta}_{\mathrm{true}}),$$

where

$$\boldsymbol{\theta}_{\mathrm{true}}$$

is a prescribed parameter vector.

To emulate realistic experimental conditions, random noise may then be added,

$$\mathbf{y}_{\mathrm{synthetic}} = f(\boldsymbol{\theta}_{\mathrm{true}}) + \boldsymbol{\varepsilon}.$$

In the special case of Gaussian noise,

$$\varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

yielding

$$y_{\mathrm{synthetic},i} = f_i(\boldsymbol{\theta}_{\mathrm{true}}) + \varepsilon_i.$$

Synthetic datasets are valuable because the true generating parameters are known exactly, enabling objective evaluation of inference algorithms and machine learning models.

4.7 Role of Synthetic Data in the Framework

Synthetic data generation serves several purposes within the Scientific AI Framework:

- creation of training datasets for neural intent and parameter prediction models,
- validation of forward simulation routines,
- benchmarking of Bayesian inference algorithms,
- verification of parameter recovery using controlled experiments,
- testing of end-to-end execution pipelines before application to experimental data.

Because the ground-truth parameters are known, synthetic datasets provide a reliable means of assessing both numerical correctness and statistical performance.

4.8 Computational Workflow

The general forward modelling process implemented throughout the framework may be summarized as follows:

1. Specify a parameter vector

$$\boldsymbol{\theta}.$$

2. Evaluate the forward model

$$\mathbf{y} = f(\boldsymbol{\theta}).$$

3. Optionally generate synthetic observations by adding measurement noise,

$$\mathbf{y}_{\text{synthetic}} = \mathbf{y} + \boldsymbol{\varepsilon}.$$

4. Return the simulated observables or use them as input for downstream tasks such as Bayesian parameter inference or machine learning.

This abstraction provides a unified mathematical formulation that applies consistently across all scientific domains implemented within the framework.

5. Hydrogen Spectrum Model

5.1 Overview

The hydrogen module models the emission spectrum of atomic hydrogen as a superposition of broadened spectral lines corresponding to electronic transitions. It supports both forward simulation, in which spectra are generated from known parameters, and Bayesian inference, in which unknown parameters are estimated from observed spectral data.

The mathematical formulation combines a deterministic forward model with probabilistic inference techniques described in the previous sections.

5.2 Spectral Line Representation

The hydrogen spectrum is constructed from electronic transitions between discrete atomic energy levels.

The energy of the hydrogen atom in the quantum state with principal quantum number n is given by

$$E_n = -\frac{R_\mathrm{H}}{n^2},$$

where

- $R_\mathrm{H} \approx 13.598285\,\mathrm{eV}$ is the Rydberg energy constant,
- $n = 1, 2, 3, \ldots$ is the principal quantum number.

For a transition from an upper level n_u to a lower level n_l, the emitted photon energy is

$$E_\mathrm{line} = E_u - E_l = -\frac{R_\mathrm{H}}{n_u^2} + \frac{R_\mathrm{H}}{n_l^2}.$$

Each transition is represented by a normalized Gaussian line profile,

$$I_i(E) = \frac{A_i}{\sigma_i\sqrt{2\pi}} \exp\left(-\frac{(E - E_{\mathrm{line},i})^2}{2\sigma_i^2}\right),$$

where

- A_i is the line amplitude,
- $E_{\mathrm{line},i}$ is the transition energy,
- σ_i is the intrinsic line width.

In the current implementation, the intrinsic line width is computed as

$$\sigma_i = \max\left(0.1,, 0.1, \sigma_\mathrm{instr}\right),$$

ensuring a minimum numerical broadening while maintaining a proportional relationship to the instrumental resolution parameter.

5.3 Instrument Resolution Parameter

Real spectrometers do not measure infinitely sharp spectral lines. Instrumental effects broaden each transition over a finite energy interval.

This broadening is represented by the parameter

$$\sigma_\mathrm{instr},$$

which is assumed to be identical for all transitions in the current implementation.

Larger values of

$$\sigma_\mathrm{instr}$$

produce wider spectral peaks, while smaller values produce sharper lines.

5.4 Background Contribution

Experimental measurements frequently contain background counts arising from detector noise, ambient radiation, or other systematic effects.

The framework models this contribution as a constant additive background,

$$B.$$

This parameter shifts the entire spectrum uniformly and is inferred jointly with the spectral amplitudes when Bayesian parameter estimation is performed.

5.5 Complete Forward Model

The intrinsic hydrogen spectrum is obtained by summing the contributions from all configured transitions,

$$I_{\mathrm{phys}}(E) = \sum_{i=1}^{N_{\mathrm{lines}}} \frac{A_i}{\sigma_i \sqrt{2\pi}} \exp\left(-\frac{(E - E_{\mathrm{line},i})^2}{2\sigma_i^2} \right).$$

To model the finite response of the measuring instrument, the intrinsic spectrum is convolved with an instrumental response function,

$$I_{\mathrm{conv}}(E) = (K * I_{\mathrm{phys}})(E),$$

where

- K denotes the instrument response kernel,
- $*$ denotes convolution.

Finally, a constant background contribution is added,

$$I(E) = I_{\mathrm{conv}}(E) + B,$$

where B is the background count level.

Negative numerical values, if produced during intermediate computations, are clipped to zero before the background contribution is applied.

This sequence—construction of intrinsic spectral lines, convolution with the instrument response, and addition of a constant background—constitutes the hydrogen forward model implemented in the framework.

5.6 Model Parameterization

For Bayesian inference, the parameter vector is represented as

$$\boldsymbol{\theta} = (\sigma_{\mathrm{instr}}, B, S, r_1, r_2, \ldots, r_N),$$

where

- σ_{instr} is the instrumental broadening,
- B is the constant background,
- S is a global intensity scale factor,
- r_i are logarithmic relative amplitudes associated with the spectral transitions.

Rather than sampling amplitudes directly, the framework computes the physical amplitudes as

$$A_i = e^{r_i}, S.$$

This parameterization guarantees strictly positive amplitudes while permitting efficient exploration of several orders of magnitude in intensity.

Very small amplitudes are clipped to positive values during numerical evaluation to avoid underflow.

5.7 Synthetic Spectrum Generation

Synthetic hydrogen spectra are generated by first evaluating the forward model using a prescribed parameter vector,

$$\boldsymbol{\theta}_{\mathrm{true}}.$$

The resulting spectrum is normalized by a global scaling factor so that its total expected intensity matches a desired count level,

$$I_{\mathrm{scaled}}(E) = C, I(E),$$

where the normalization constant C is chosen such that

$$\sum_i I_{\mathrm{scaled}}(E_i) \approx 10^5$$

in the current implementation.

A constant background contribution is then added to every energy bin.

The observed photon counts are generated by independent Poisson sampling,

$$k_i \sim \mathrm{Poisson!}\left(I_{\mathrm{scaled}}(E_i)\right),$$

where k_i denotes the measured count in the i-th energy bin.

The associated statistical uncertainty is estimated using the standard Poisson approximation,

$$\sigma_i = \sqrt{\max(k_i, 1)},$$

which avoids zero-valued uncertainties for bins containing no observed counts.

The synthetic datasets generated by this procedure provide realistic photon-counting spectra with known ground-truth parameters and are used for validation, benchmarking, and testing of Bayesian inference algorithms.

5.8 Prior Distributions

The hydrogen inference module combines hard parameter constraints with probabilistic prior distributions.

Parameter Bounds

The implementation restricts physically meaningful parameter values to predefined intervals. Parameter vectors outside these bounds are assigned zero prior probability,

$$P(\boldsymbol{\theta}) = 0,$$

or equivalently,

$$\log P(\boldsymbol{\theta}) = -\infty.$$

This prevents the Markov Chain Monte Carlo sampler from exploring invalid regions of parameter space.

Prior for Global Scale

The global intensity scale parameter

$$S$$

is assigned a log-normal prior,

$$\log S \sim \mathcal{N}!\left(\mu, \sigma_{\log}^2\right),$$

where the implementation uses

$$\mu = \log(10^4)$$

and

$$\sigma_{\log} = 3.$$

This prior permits several orders of magnitude variation while favouring physically reasonable scales.

Prior for Instrumental Broadening

The instrumental broadening parameter is assigned an exponential prior,

$$\sigma_{\mathrm{instr}} \sim \mathrm{Exponential}(\beta),$$

with scale parameter

$$\beta = 10.$$

This prior favours smaller broadening values while allowing occasional larger values when supported by the data.

5.9 Likelihood Function

Observed spectra are treated as photon-counting measurements.

If

- k_i denotes the observed count in energy bin i,
- λ_i denotes the corresponding model prediction,

then the counts are modelled using a Poisson distribution,

$$k_i \sim \mathrm{Poisson}(\lambda_i).$$

The probability of observing

$$k_i$$

counts is

$$P(k_i \mid \lambda_i) = \frac{\lambda_i^{k_i} e^{-\lambda_i}}{k_i!}.$$

Assuming independence between energy bins, the total likelihood is

$$P(D \mid \boldsymbol{\theta}) = \prod_i \frac{\lambda_i^{k_i} e^{-\lambda_i}}{k_i!}.$$

For numerical stability, the implementation evaluates the corresponding log-likelihood,

$$\log P(D \mid \boldsymbol{\theta}) = \sum_i \left[k_i \log \lambda_i - \lambda_i \log(k_i!) \right].$$

Predicted counts are constrained to remain positive before evaluating the logarithm in order to avoid numerical singularities.

5.10 Posterior Distribution

The posterior probability distribution combines the prior and likelihood according to Bayes' theorem,

$$P(\boldsymbol{\theta} \mid D) \propto P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta}).$$

Equivalently, in logarithmic form,

$$\log P(\boldsymbol{\theta} \mid D) = \log P(D \mid \boldsymbol{\theta}) + \log P(\boldsymbol{\theta}).$$

This log-posterior function constitutes the objective evaluated repeatedly during Markov Chain Monte Carlo sampling.

5.11 Bayesian Parameter Inference

Given an observed hydrogen spectrum, the objective is to estimate the unknown parameter vector

$$\boldsymbol{\theta} = (\sigma_{\text{instr}}, B, S, r_1, r_2, \ldots, r_N).$$

The framework evaluates the forward model for proposed parameters, computes the corresponding Poisson log-likelihood, incorporates the prior distributions, and samples the resulting posterior using Markov Chain Monte Carlo methods.

The retained posterior samples provide estimates of parameter values together with credible intervals and uncertainty quantification.

5.12 Mapping to Implementation Variables

Mathematical Symbol	Description	Typical Implementation Variable
E	Energy grid	`energies`
E_i	Transition energies	`transitions`
A_i	Physical line amplitudes	`amplitudes`
r_i	Log-relative amplitudes	`line_rel`
S	Global scale factor	`scale`
σ_{instr}	Instrument resolution parameter	`sigma_instr`
B	Constant background	`background`
$I(E)$	Simulated spectrum	Forward model output
k_i	Observed counts	`data.counts`
λ_i	Model-predicted counts	`model`
$\boldsymbol{\theta}$	Inference parameter vector	`theta`

The mathematical formulation presented above directly corresponds to the implementation of the hydrogen forward model and Bayesian inference routines in the Scientific AI Framework.

6. Single-Qubit Quantum Dynamics Model

6.1 Overview

The single-qubit module models the coherent time evolution of a two-level quantum system undergoing driven oscillations in the presence of detuning and exponential decoherence. The resulting signal is represented as a damped cosine function and serves as the basis for both forward simulation and Bayesian parameter inference.

The model predicts a time-dependent observable measured at discrete sampling times.

6.2 Effective Rabi Frequency

The dynamics are governed by the applied Rabi frequency

$$\Omega_R$$

and the frequency detuning

$$\Delta.$$

The corresponding effective oscillation frequency is

$$\Omega_{\text{eff}} = \sqrt{\Omega_R^2 + \Delta^2}.$$

This quantity determines the oscillation frequency observed in the measured signal.

6.3 Forward Model

Let

- t denote time,
- A denote the oscillation amplitude,
- γ denote the exponential decay rate,
- C denote a constant offset,
- Ω_{eff} denote the effective oscillation frequency.

The predicted measurement is

$$y(t) = C + Ae^{-\gamma t} \cos\left(\Omega_{\text{eff}} t\right).$$

Substituting the expression for the effective frequency,

$$y(t) = C + Ae^{-\gamma t} \cos\left(\sqrt{\Omega_R^2 + \Delta^2}\, t\right).$$

The model consists of four components:

1. a constant baseline offset,
2. an oscillatory cosine term,
3. exponential damping,
4. an oscillation frequency determined jointly by the Rabi frequency and detuning.

6.4 Model Parameters

The forward model depends on the parameter vector

$$\boldsymbol{\theta} = (\Omega_R, \Delta, A, \gamma, C),$$

where

- Ω_R is the Rabi frequency,
- Δ is the detuning,
- A is the oscillation amplitude,
- γ is the exponential decay constant,
- C is the constant measurement offset.

For fixed parameter values and sampling times, the forward model is deterministic.

6.5 Time Grid

The model is evaluated on a collection of discrete sampling times

$$t_1, t_2, \ldots, t_N.$$

The predicted measurement vector is therefore

$$\mathbf{y} = \left(y(t_1), y(t_2), \ldots, y(t_N)\right).$$

These predicted values are subsequently compared with observed measurements during Bayesian inference.

6.6 Synthetic Data Generation

Synthetic datasets are generated by first evaluating the noiseless forward model,

$$y_{\text{clean}}(t) = C + Ae^{-\gamma t}\cos\left(\Omega_{\text{eff}} t\right),$$

using prescribed ground-truth parameters.

Independent Gaussian measurement noise is then added,

$$\varepsilon_i \sim \mathcal{N}\left(0, \sigma_{\text{noise}}^2\right),$$

yielding the synthetic observations

$$y_i = y_{\text{clean}}(t_i) + \varepsilon_i.$$

The standard deviation of the measurement uncertainty is assumed constant,

$$\sigma_i = \sigma_{\text{noise}},$$

for every sampling time.

6.7 Noise Model

The current implementation assumes additive independent Gaussian noise with zero mean,

$$\varepsilon_i \sim \mathcal{N}\left(0, \sigma_{\text{noise}}^2\right).$$

Consequently,

$$y_i \sim \mathcal{N}\left(y_{\text{clean}}(t_i), \sigma_{\text{noise}}^2\right).$$

This model is appropriate for many continuous-valued laboratory measurements in which detector fluctuations are approximately normally distributed.

6.8 Mapping to Implementation Variables

Mathematical Symbol	Description	Implementation Variable
t	Time samples	`times`
Ω_R	Rabi frequency	`omega_r`
Δ	Detuning	`detuning`
Ω_{eff}	Effective oscillation frequency	`omega_eff`
A	Oscillation amplitude	`amp`
γ	Exponential decay constant	`gamma`
C	Constant offset	`offset`
σ_{noise}	Measurement noise standard deviation	`noise_std`
$y(t)$	Predicted measurement	`measurements_clean`
y_i	Synthetic observation	`measurements`
σ_i	Measurement uncertainty	`errors`

The equations presented above directly correspond to the implementation of the single-qubit forward model and synthetic data generator within the Scientific AI Framework.

6.9 Bayesian Parameter Inference

The single-qubit inference module estimates the model parameters from measured time-series data using Bayesian inference. The unknown parameter vector is

$$\boldsymbol{\theta} = (\Omega_R, \Delta, \gamma, A, C),$$

where

- Ω_R is the Rabi frequency,
- Δ is the detuning,
- γ is the exponential decay constant,
- A is the oscillation amplitude,
- C is the constant measurement offset.

Given observed measurements, the objective is to estimate the posterior distribution

$$P(\boldsymbol{\theta} \mid D),$$

where D denotes the measured time-series data.

6.10 Prior Distributions

The inference procedure combines hard parameter bounds with probabilistic prior distributions.

Hard Parameter Bounds

To restrict sampling to physically meaningful regions, the implementation enforces the following bounds:

$$0 \leq \Omega_R \leq 10^3,$$

$$-10^3 \le \Delta \le 10^3,$$

$$0 \le \gamma \le 10^2,$$

$$-10^6 \le A \le 10^6,$$

$$-10^6 \le C \le 10^6.$$

Parameter vectors violating any of these constraints are assigned zero prior probability,

$$P(\boldsymbol{\theta}) = 0,$$

or equivalently,

$$\log P(\boldsymbol{\theta}) = -\infty.$$

This prevents the Markov Chain Monte Carlo sampler from exploring invalid regions of parameter space.

Exponential Prior for the Decay Constant

The decay parameter is assigned an exponential prior,

$$\gamma \sim \mathrm{Exponential}(\beta),$$

where the implementation uses

$$\beta = 1.$$

The corresponding probability density is

$$P(\gamma) = \frac{1}{\beta} \exp\left(-\frac{\gamma}{\beta}\right), \qquad \gamma \ge 0.$$

For $\beta = 1$, this simplifies to

$$P(\gamma) = e^{-\gamma}, \qquad \gamma \ge 0.$$

This prior favors slower decoherence while permitting larger decay rates when supported by the observed data.

6.11 Likelihood Function

Let

$$y_i$$

denote the measured value at sampling time

$$t_i,$$

and let

$$f(t_i; \boldsymbol{\theta})$$

denote the corresponding prediction of the forward model.

The residual is defined as

$$r_i = y_i f(t_i; \boldsymbol{\theta}).$$

The implementation assumes additive independent Gaussian measurement errors with known standard deviations

$$\sigma_i.$$

Accordingly,

$$y_i \sim \mathcal{N}\left(f(t_i;\boldsymbol{\theta}), \sigma_i^2\right).$$

The implementation evaluates the Gaussian log-likelihood in the numerically stable form

$$\log P(D \mid \boldsymbol{\theta}) = -\frac{1}{2}\sum_{i=1}^{N}\left[\frac{r_i^2}{\sigma_i^2} + \log\left(2\pi\sigma_i^2\right)\right],$$

where

$$r_i = y_i - f(t_i;\boldsymbol{\theta})$$

denotes the residual between the observed measurement and the model prediction at the i-th sampling time.

To avoid numerical instabilities, the variances are constrained to remain strictly positive before evaluating the logarithm.

6.12 Posterior Distribution

The posterior distribution is obtained by combining the prior and likelihood according to Bayes' theorem,

$$P(\boldsymbol{\theta} \mid D) \propto P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta}).$$

Equivalently, in logarithmic form,

$$\log P(\boldsymbol{\theta} \mid D) = \log P(D \mid \boldsymbol{\theta}) + \log P(\boldsymbol{\theta}).$$

This log-posterior function is the quantity evaluated repeatedly during Markov Chain Monte Carlo sampling.

6.13 Posterior Summaries

After sampling, an initial burn-in fraction of the Markov chains is discarded to reduce dependence on the starting positions of the walkers.

The retained samples are then used to compute summary statistics of the posterior distribution.

Maximum a Posteriori Estimate

The Maximum a Posteriori (MAP) estimate is defined as the sampled parameter vector with the highest posterior probability,

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} P(\boldsymbol{\theta} \mid D).$$

The MAP estimate represents the single most probable parameter vector identified by the sampler.

Posterior Mean

The posterior mean is computed as the arithmetic average of the retained samples,

$$\bar{\boldsymbol{\theta}} = \frac{1}{N}\sum_{k=1}^{N} \boldsymbol{\theta}^{(k)},$$

where

$$\boldsymbol{\theta}^{(k)}$$

denotes the k-th posterior sample.

The posterior mean incorporates information from the entire sampled distribution and is often less sensitive to sampling variability than a single point estimate.

6.14 Computational Workflow

The Bayesian inference procedure implemented for the single-qubit model follows the sequence:

1. Define the parameter vector
$$\boldsymbol{\theta} = (\Omega_R, \Delta, \gamma, A, C).$$
2. Evaluate the forward model to predict the measured signal.
3. Compute the Gaussian log-likelihood from the residuals between predicted and observed measurements.
4. Evaluate the log-prior, including hard parameter bounds and the exponential prior on the decay constant.
5. Form the log-posterior by summing the log-likelihood and log-prior.
6. Use Markov Chain Monte Carlo sampling to generate samples from the posterior distribution.
7. Discard burn-in samples and compute posterior summaries, including the Maximum a Posteriori estimate and posterior mean.

This methodology provides statistically principled estimates of the single-qubit model parameters together with associated uncertainty quantification.

7. Multi-Qubit Quantum Dynamics Model

7.1 Overview

The multi-qubit module models the coherent evolution of an interacting quantum system consisting of multiple two-level subsystems (qubits). The dynamics are governed by a Hamiltonian constructed from local control fields and pairwise qubit couplings.

The framework supports both time-domain evolution and frequency-domain spectrum generation using the same underlying Hamiltonian.

7.2 Hilbert Space Representation

For a system containing

$$N$$

qubits, the total Hilbert space is

$$\mathcal{H} = (\mathbb{C}^2)^{\otimes N},$$

where

$$\otimes$$

denotes the tensor product.

Each qubit is represented by a two-dimensional state space, and operators acting on individual qubits are embedded into the full Hilbert space through tensor products with identity operators acting on the remaining qubits.

7.3 Pauli Operators

For each qubit, the standard Pauli matrices

$$\sigma_x, \qquad \sigma_y, \qquad \sigma_z$$

are employed as the fundamental observables and generators of evolution.

The implementation constructs tensor-product operators of the form

$$\sigma_\alpha^{(i)}, \qquad \alpha \in x, y, z,$$

which act only on qubit

$$i$$

while leaving all remaining qubits unchanged.

7.4 Hamiltonian Construction

For each qubit

$$i,$$

the local control fields are specified by the parameters

$$h_x^{(i)}, \qquad h_y^{(i)}, \qquad h_z^{(i)}.$$

The local contribution to the Hamiltonian is

$$H_{\text{local}} = \sum_{i=1}^{N} \left(h_x^{(i)} \sigma_x^{(i)} + h_y^{(i)} \sigma_y^{(i)} + h_z^{(i)} \sigma_z^{(i)} \right).$$

In the current implementation, the optional qubit frequency parameter is stored but is not included in the Hamiltonian construction.

7.5 Pairwise Couplings

Interactions between qubits are represented by pairwise coupling constants

$$g_{ij}.$$

For coupled qubits

$$i$$

and

$$j,$$

the interaction Hamiltonian is

$$H_{\text{coupling}} = \sum_{i<j} g_{ij}, \sigma_z^{(i)} \sigma_z^{(j)}.$$

The implementation supports specification of couplings as a global constant, a dictionary of qubit pairs, or an explicit list of coupling tuples.

7.6 Total Hamiltonian

The complete Hamiltonian governing the system evolution is

$$H = H_{\text{local}} + H_{\text{coupling}}.$$

Operationally, the software constructs individual Hamiltonian terms and sums them once to obtain the final Hamiltonian operator used for simulation.

7.7 Initial Quantum State

The simulations begin from a product state in which the first qubit is initialized in the excited computational basis state and all remaining qubits are initialized in the ground state,

$$|\psi_0\rangle = |1\rangle \otimes |0\rangle \otimes \cdots \otimes |0\rangle.$$

This state serves as the initial condition for subsequent time evolution.

7.8 Schrödinger Time Evolution

The time evolution of the quantum state satisfies the time-dependent Schrödinger equation,

$$i, \frac{\partial}{\partial t}|\psi(t)\rangle = H|\psi(t)\rangle,$$

where

- H is the Hamiltonian,
- $|\psi(t)\rangle$ is the system state at time t.

The framework numerically integrates this equation to obtain the quantum state at all requested sampling times.

7.9 Measurement Observables

For each qubit, the measured observable is the expectation value of the Pauli-Z operator,

$$\langle \sigma_z^{(i)} \rangle = \langle \psi(t) | \sigma_z^{(i)} | \psi(t) \rangle.$$

Collecting these expectation values over time produces a matrix of predicted measurements with one column per qubit and one row per sampling instant.

7.10 Instrument Model

After quantum evolution, the simulated observables are transformed by a simple linear instrument model,

$$y_{\text{measured}} = G * y_{\text{quantum}} + C,$$

where

- G is the instrument gain,
- C is a constant DC offset.

These transformed values constitute the final predicted measurements returned by the forward model.

7.11 Spectrum Mode

In frequency-domain operation, the Hamiltonian eigenvalues

$$E_1, E_2, \ldots, E_M$$

are computed by diagonalization.

Allowed transition energies are obtained from pairwise eigenvalue differences,

$$\Delta E_{ij} = E_i - E_j, \qquad i > j.$$

Each transition contributes a Gaussian peak to the simulated spectrum,

$$S(f) = \sum_k \exp\left(-\frac{(f - \Delta E_k)^2}{2\sigma_{\text{instr}}^2}\right),$$

where

$$\sigma_{\text{instr}}$$

is the instrumental broadening parameter.

The spectrum is normalized by its maximum value before the addition of a constant background level,

$$S_{\text{final}}(f) = B + \frac{S(f)}{\max S(f)},$$

where

$$B$$

denotes the background contribution.

7.12 Mapping to Implementation Variables

Mathematical Symbol	Description	Implementation Variable
H	Total Hamiltonian	`H`
h_x, h_y, h_z	Local control fields	`h_x`, `h_y`, `h_z`
g_{ij}	Pairwise coupling strength	`couplings`
$\sigma_x, \sigma_y, \sigma_z$	Pauli operators	`qt.sigmax()`, `qt.sigmay()`, `qt.sigmaz()`
$\langle \psi_0 \rangle$	Initial quantum state	`psi0`
$\langle \psi(t) \rangle$	Time-evolved quantum state	`qt.sesolve` solution
$\langle \sigma_z \rangle$	Measured expectation values	`result.expect`
G	Instrument gain	`gain`

Mathematical Symbol	Description	Implementation Variable
C	DC offset	`dc_offset`
σ_{instr}	Spectral broadening	`sigma_instr`
B	Spectrum background	`background`

The mathematical formulation above corresponds directly to the Hamiltonian construction, Schrödinger evolution, observable evaluation, and spectrum-generation procedures implemented in the multi-qubit forward model.

7.13 Synthetic Data Generation

Synthetic multi-qubit datasets are generated by numerically evolving a quantum system under a user-specified Hamiltonian and recording expectation values of selected observables.

The procedure consists of the following steps.

Hamiltonian Construction

Given the number of qubits, local control fields, and pairwise couplings, a symbolic Hamiltonian

$$H$$

is constructed according to the mathematical formulation described in the preceding sections.

The symbolic representation is subsequently converted into a numerical matrix suitable for simulation.

Initial State Preparation

If no user-defined initial state is supplied, the framework constructs a default product state

$$|\psi_0\rangle,$$

which serves as the starting point for quantum evolution.

Alternatively, an arbitrary user-provided initial state may be used.

Observable Construction

For each requested observable specification, an operator

$$O_i$$

is constructed on the full Hilbert space.

Typical observables correspond to tensor-product extensions of the Pauli operators acting on individual qubits.

Quantum Evolution

Let

$$t_1, t_2, \ldots, t_N$$

denote the requested sampling times.

The framework evolves the initial state according to either

- **unitary evolution**, governed by the Schrödinger equation,

$$i\frac{\partial}{\partial t}|\psi(t)\rangle = H|\psi(t)\rangle,$$

or

- **open-system evolution**, when collapse operators are supplied, using an appropriate Lindblad-type evolution model.

The choice between these two modes is controlled by the configuration parameters.

Measurement Generation

For each observable

$$O_i,$$

the synthetic measurement at time

$$t$$

is computed as the quantum expectation value

$$m_i(t) = \langle\psi(t)|O_i|\psi(t)\rangle.$$

Collecting all observables over all sampling times produces the synthetic measurement matrix

$$M = \begin{bmatrix} m_1(t_1) & \cdots & m_1(t_N) \\ m_2(t_1) & \cdots & m_2(t_N) \\ \vdots & \ddots & \vdots \\ m_K(t_1) & \cdots & m_K(t_N) \end{bmatrix},$$

where

$$K$$

is the number of measured observables.

Deterministic Synthetic Data

Unlike the single-qubit synthetic generator, the current multi-qubit implementation does **not** add random measurement noise to the simulated observables.

Consequently, the returned measurements correspond directly to the expectation values predicted by the underlying quantum evolution,

$$y_{\text{synthetic}} = m(t),$$

without an additional stochastic perturbation term.

This deterministic formulation provides reproducible benchmark datasets that are particularly useful for validating forward simulations and Bayesian inference algorithms.

Metadata

In addition to the simulated measurements, the generated dataset stores metadata describing the simulation configuration, including

- the number of qubits,

- local control fields,
- pairwise coupling parameters,
- observable specifications,
- Hamiltonian parameters,
- and whether unitary or open-system evolution was employed.

These metadata facilitate reproducibility and downstream parameter inference.

7.14 Bayesian Inference

Bayesian parameter estimation for the multi-qubit model follows the same framework described in Sections 2 and 6. In particular,

- the posterior distribution is obtained from Bayes' theorem,
- the log-posterior is computed as the sum of the log-prior and log-likelihood,
- Markov Chain Monte Carlo sampling is employed to explore the posterior distribution,
- posterior summaries include both the Maximum a Posteriori (MAP) estimate and the posterior mean.

Multivariate Gaussian Prior

Unlike the Hydrogen and Single-Qubit models, the Multi-Qubit implementation employs a correlated multivariate Gaussian prior over the parameter vector,

$$\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \Sigma),$$

with covariance matrix

$$\Sigma = 0.1\, I + 0.009\, J,$$

where I is the identity matrix and J is the matrix of all ones. The resulting log-prior is

$$\log P(\boldsymbol{\theta}) = -\frac{1}{2}(\boldsymbol{\theta})^T \Sigma^{-1} \boldsymbol{\theta} - \frac{1}{2}\log|\Sigma| - \frac{n}{2}\log(2\pi).$$

Likelihood Function

The predicted observables produced by the forward quantum simulation are flattened into a single vector and compared with the measured data under an additive Gaussian noise model. Assuming a constant noise standard deviation σ, the log-likelihood is

$$\log P(D \mid \boldsymbol{\theta}) = -\frac{1}{2}\left[\frac{\sum_i (y_i^{\text{obs}} - y_i^{\text{pred}})^2}{\sigma^2} + N\log(2\pi\sigma^2)\right].$$

The remainder of the Bayesian inference procedure, including posterior construction, MCMC sampling, burn-in removal, MAP estimation, and posterior mean computation, is identical to the methodology described for the Single-Qubit model.

7.15 Computational Scaling

An N-qubit system is represented in a Hilbert space of dimension

$$\dim(\mathcal{H}) = 2^N.$$

Consequently,

- state vectors contain 2^N complex amplitudes,
- Hamiltonian matrices have dimensions $2^N \times 2^N$,

- and the computational cost of dense simulation grows exponentially with the number of qubits.

For this reason, practical simulations are limited to moderate system sizes unless specialized sparse or tensor-network techniques are employed.

The Scientific AI Framework therefore includes safeguards on the maximum number of qubits used in dense simulations to avoid excessive computational cost.

9. Loss Functions Used During Training

The neural intent model is trained using a multi-task objective that combines classification losses and masked regression losses. Different output heads correspond to different prediction tasks, and the total loss is formed by aggregating their individual contributions.

9.1 Cross-Entropy Loss for Classification

Several output heads perform categorical prediction tasks, including domain identification, action prediction, spectrum mode classification, evolution mode classification, initial state selection, observable prediction, topology prediction, and open-system classification.

For a classification problem with true class label (y) and predicted logits (\mathbf{z}), the cross-entropy loss is

$$L_{\mathrm{CE}} = \log\left(\frac{\exp(z_y)}{\sum_j \exp(z_j)}\right).$$

Equivalently,

$$L_{\mathrm{CE}} = \sum_j y_j \log p_j,$$

where

$$p_j = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

is the predicted probability obtained through the softmax transformation.

The framework applies this loss independently to each categorical prediction head.

9.2 Masked Mean Squared Error

Continuous-valued regression targets are trained using a masked mean squared error (MSE).

Let

- (t_i) denote the target value,
- (\hat{t}_i) denote the predicted value,
- (m_i \in {0,1}) denote the corresponding regression mask.

The squared error for each regression component is

$$(\hat{t}_i - t_i)^2.$$

Applying the mask yields

$$m_i(\hat{t}_i - t_i)^2.$$

The overall masked regression loss is

$$L_{\text{reg}} = \frac{\sum_i m_i(\hat{t}_i - t_i)^2}{\sum_i m_i + \varepsilon},$$

where

$$\varepsilon = 10^{-8}$$

prevents division by zero when no active regression targets are present.

Only active regression slots therefore contribute to the optimization objective.

9.3 Masked Cross-Entropy for Optional Outputs

Certain categorical outputs are relevant only for specific training examples.

For these heads, a binary mask is applied after computing the per-sample cross-entropy loss,

$$L_{\text{masked}} = \frac{\sum_i m_i L_i}{\sum_i m_i + \varepsilon},$$

where

- (L_i) is the cross-entropy for sample (i),
- (m_i) indicates whether the corresponding label is active.

Inactive labels therefore make no contribution to the optimization.

9.4 Combined Training Objective

The complete training loss combines multiple classification losses together with the masked regression objective.

Conceptually,

$$L_{\text{total}} = L_{\text{domain}} + L_{\text{action}} + L_{\text{spectrum}} + L_{\text{series}} + L_{\text{evolution}} + L_{\text{initial}} + L_{\text{observable}} + L_{\text{topology}} + L_{\text{open}} + \lambda_{\text{reg}} L_{\text{reg}},$$

where

$$\lambda_{\text{reg}}$$

controls the relative contribution of the regression component.

In the present implementation,

$$\lambda_{\text{reg}} = 0.3.$$

This multi-task formulation enables simultaneous optimization of both discrete intent classification and continuous parameter prediction.

10. Regression Normalization

Continuous regression parameters are standardized using statistics computed from the training dataset. This normalization places heterogeneous physical quantities on comparable numerical scales, improving optimization

stability and learning efficiency.

10.1 Computation of Dataset Statistics

For each continuous regression slot, let

$$x_1, x_2, \ldots, x_N$$

denote the observed training values.

The empirical mean is computed as

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i,$$

and the empirical standard deviation is

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2 + 10^{-8}},$$

where the small constant

$$10^{-8}$$

prevents numerical instability when the variance is very small.

The resulting statistics are stored for each regression slot and serialized for reuse.

10.2 Standardization

Given a regression value x, the normalized quantity is obtained through z-score standardization,

$$x_{\text{norm}} = \frac{x - \mu}{\sigma},$$

where

- μ is the empirical mean for that regression slot,
- σ is the corresponding empirical standard deviation.

Each regression slot is normalized independently using its own statistics.

10.3 De-normalization

For interpretation and evaluation in physical units, normalized predictions are transformed back to their original scale.

If

$$x_{\text{norm}}$$

denotes the normalized prediction, the recovered physical value is

$$x = x_{\text{norm}}\, \sigma + \mu.$$

The framework applies this transformation independently to both predicted and target regression values before computing evaluation metrics such as the mean absolute error (MAE).

10.4 Masked Error Evaluation

Only regression slots that are active for a given sample contribute to the reported error metrics.

Let

- $m_i \in \{0, 1\}$ denotes the regression mask,
- x_i denotes the de-normalized target,
- $\hat{x}_i$ denotes the corresponding de-normalized prediction.

The masked mean absolute error is

$$\mathrm{MAE} = \frac{\sum_i m_i \, |\hat{x}_i - x_i|}{\sum_i m_i + 10^{-8}}.$$

Inactive regression slots therefore do not influence the reported evaluation statistics.

10.5 Diagnostic Statistics

The preprocessing utilities additionally compute and report

- empirical means,
- empirical standard deviations,
- minimum and maximum values,
- regression mask activation rates,
- and simple three-standard-deviation outlier counts.

These diagnostics assist in validating dataset quality and ensuring that the normalization statistics are representative of the underlying training distribution.

11. Neural Intent Detection

The Scientific AI Framework employs a character-level neural architecture to map natural language queries directly to structured scientific intents. Rather than relying solely on rule-based parsing, the model learns a distributed representation of input text and predicts both categorical intent labels and continuous numerical parameters.

11.1 Character-Level Encoding

Let an input query be represented as a sequence of characters

$$c_1, c_2, \ldots, c_T,$$

where

$$T$$

denotes the sequence length.

Each character is mapped to an integer index through a vocabulary lookup and subsequently converted into a dense embedding vector.

11.2 Embedding Layer

For a vocabulary of size

$$V,$$

each character index is transformed into an embedding

$$\mathbf{e}_t \in \mathbb{R}^d,$$

where

$$d$$

is the embedding dimension.

Collectively, the embedding matrix is

$$E \in \mathbb{R}^{V \times d},$$

and the embedded input sequence is

$$\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_T.$$

The embedding vectors are learned jointly with the remainder of the neural network during training.

11.3 Recurrent Sequence Encoder

The embedded sequence is processed sequentially by a recurrent neural network.

At each time step,

$$\mathbf{h}_t = f\left(\mathbf{e}_t, \mathbf{h}_{t-1}\right),$$

where

- $\mathbf{h}_t$ is the hidden state,
- $\mathbf{e}_t$ is the current character embedding,
- and $f(\cdot)$ denotes the recurrent update implemented by the encoder.

After processing the complete input sequence, the final hidden representation

$$\mathbf{h}_T$$

serves as a fixed-length encoding of the entire query.

11.4 Shared Latent Representation

The final hidden state

$$\mathbf{h}_T$$

acts as a learned semantic representation that captures the intent expressed by the input text.

This shared representation is subsequently supplied to multiple prediction heads, enabling simultaneous estimation of discrete and continuous outputs.

11.5 Multi-Layer Perceptron

The encoder output is transformed through a feed-forward neural network,

$$\mathbf{z} = g(\mathbf{h}_T),$$

where

$$g(\cdot)$$

denotes the multilayer perceptron (MLP).

The resulting latent vector provides the common feature representation used by all downstream prediction tasks.

11.6 Classification Heads

Separate output heads predict categorical quantities such as

- scientific domain,
- requested action,
- spectrum mode,
- evolution mode,
- observable type,
- topology,
- initial state,
- and open-system configuration.

For a classification head with logits

$$\mathbf{s},$$

the predicted probabilities are obtained through the softmax transformation,

$$P(y = i) = \frac{\exp(s_i)}{\sum_j \exp(s_j)}.$$

The predicted class corresponds to the maximum-probability category.

11.7 Regression Heads

Continuous physical parameters are predicted through dedicated regression outputs.

Let

$$\mathbf{r} = (r_1, r_2, \ldots, r_K)$$

denote the vector of regression predictions for

$$K$$

continuous slots.

Each regression component estimates a normalized physical quantity associated with the inferred scientific intent, including parameters such as frequencies, amplitudes, coupling strengths, instrumental broadening, and background levels.

The regression outputs are subsequently transformed back to physical units using the de-normalization procedure described in Section 10.

11.8 Multi-Task Learning

The architecture jointly optimizes all prediction heads using a shared latent representation.

This multi-task formulation enables information learned for one prediction task (for example, domain identification) to improve related tasks such as action prediction or parameter estimation.

Consequently, the neural intent detector simultaneously performs

- sequence understanding,
- categorical intent classification,
- and continuous parameter regression

within a unified end-to-end trainable model.

12. Training Pipeline and Optimization

The Scientific AI Framework employs a multi-task neural architecture that simultaneously predicts high-level scientific intents and the numerical parameters required for downstream computations. The training procedure combines classification and regression objectives within a unified optimization framework while accommodating heterogeneous parameter sets across multiple scientific domains.

12.1 Multi-Task Prediction Architecture

Given an input query represented by tokenized embeddings

$$\mathbf{x} = (x_1, x_2, \ldots, x_n),$$

the transformer encoder produces a contextual representation

$$\mathbf{h} = f_{\text{Transformer}}(\mathbf{x}).$$

From this shared representation, independent output heads predict:

- Scientific domain,
- Requested action,
- Categorical scientific slots,
- Continuous-valued regression parameters.

Formally,

$$\mathbf{h} \longrightarrow \left(\hat{d}, \hat{a}, \hat{\mathbf{c}}, \hat{\mathbf{r}}\right),$$

where

- $\hat{d}$ denotes the predicted domain,
- $\hat{a}$ denotes the predicted action,
- $\hat{\mathbf{c}}$ denotes categorical slot predictions,
- $\hat{\mathbf{r}}$ denotes continuous regression outputs.

The shared encoder allows information learned from one task to improve representations used by the others.

12.2 Global Regression Slot Vector

Instead of constructing a different regression head for every scientific model, the framework defines a single global regression vector

$$\hat{\mathbf{r}} = (\hat{r}_1, \hat{r}_2, \ldots, \hat{r}_S),$$

where S is the total number of supported regression slots across all domains.

Only a subset of these parameters is relevant for any particular user query. For example:

- Hydrogen spectrum inference requires parameters such as transition energies, amplitudes, instrumental broadening, and background.
- Single-qubit simulations require Hamiltonian coefficients and evolution parameters.
- Multi-qubit systems require coupling constants and additional quantum-mechanical parameters.

Consequently, many components of the global regression vector are intentionally unused for a given training example.

12.3 Regression Masking

To prevent irrelevant parameters from influencing optimization, each sample is associated with a binary regression mask

$$\mathbf{m} = (m_1, m_2, \ldots, m_S), \qquad m_i \in 0, 1.$$

The mask indicates whether regression slot i is applicable to the current scientific intent.

The masked mean squared error is

$$L_{\text{reg}} = \frac{\sum_{i=1}^{S} m_i (\hat{r}_i - r_i)^2}{\sum_{i=1}^{S} m_i},$$

or, for batched training,

$$L_{\text{reg}} = \frac{\sum_b \sum_i m_{b,i} (\hat{r}_{b,i} - r_{b,i})^2}{\sum_b \sum_i m_{b,i}}.$$

Only active regression slots contribute to the optimization objective, while inactive slots produce zero loss and zero gradient.

12.4 Normalized Regression Targets

Regression targets often possess widely different numerical scales. To stabilize optimization, each regression variable is transformed into a normalized representation prior to training.

The neural network therefore predicts normalized quantities rather than raw physical parameters.

Training minimizes the masked regression loss using these normalized targets, ensuring that variables with large physical magnitudes do not dominate the optimization process.

After prediction, outputs may be transformed back into physical units using stored normalization statistics for reporting and evaluation purposes. This denormalization step is used only for metric computation (such as mean absolute error in physical units) and does not participate in gradient computation or parameter updates.

12.5 Multi-Task Objective Function

The overall training objective combines multiple supervised tasks into a single scalar loss.

Let

- L_{domain} denote domain classification loss,
- L_{action} denote action classification loss,
- $L_{\text{cat}}^{(k)}$ denote the losses for categorical scientific slots,

- L_{reg} denote the masked regression loss.

The total objective may be expressed as

$$L_{\text{total}} = L_{\text{domain}} + L_{\text{action}} + \sum_{k} L_{\text{cat}}^{(k)} + \lambda_{\text{reg}} L_{\text{reg}},$$

where λ_{reg} controls the relative contribution of the regression task.

The optimization procedure minimizes this combined objective using gradient-based learning.

12.6 Mini-Batch Optimization

Training proceeds over mini-batches of examples. For each batch:

1. The optimizer gradients are reset.
2. The transformer performs a forward pass.
3. Classification and regression losses are computed.
4. The combined loss is backpropagated.
5. Model parameters are updated using the optimizer.

This iterative process approximates minimization of the empirical risk over the training dataset.

12.7 Gradient Clipping

Large gradients may occasionally arise during optimization, particularly in deep transformer architectures or multi-task learning settings.

To improve numerical stability, the framework clips parameter gradients according to

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{\tau}{|\mathbf{g}|}\right),$$

where

- $\mathbf{g}$ denotes the gradient vector,
- $|\mathbf{g}|$ is its Euclidean norm,
- τ is the clipping threshold.

This operation preserves gradient direction while preventing excessively large parameter updates.

12.8 Evaluation Metrics

During evaluation, regression predictions are transformed back into their corresponding physical units before computing performance metrics such as the mean absolute error (MAE),

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} \left| \hat{r}_i - r_i \right|.$$

Because these computations are detached from the computational graph, they do not influence optimization and serve solely as interpretable measures of predictive accuracy.

13. Execution Pipeline

The Scientific AI Framework separates **natural language understanding** from **scientific computation**. The neural network is responsible for interpreting the user's query and predicting the required structured information, while

specialized domain executors perform the underlying numerical calculations using established physical and mathematical models.

The overall execution pipeline may be represented as

```
User Query
      ↓
Neural Intent Detector
      ↓
Intent and Slot Prediction
      ↓
ScientificIntent
      ↓
Domain Executor
      ↓
Forward Model or Bayesian Inference
      ↓
Numerical Results
```

13.1 Natural Language Interpretation

Let the user query be represented as a sequence of tokens

$$\mathbf{x} = (x_1, x_2, \ldots, x_n).$$

The transformer encoder computes contextual embeddings

$$\mathbf{h} = f_{\text{Transformer}}(\mathbf{x}),$$

from which the prediction heads estimate

- the scientific domain,
- the requested action,
- categorical slot values, and
- continuous regression parameters.

Mathematically,

$$(\hat{d}, \hat{a}, \hat{\mathbf{c}}, \hat{\boldsymbol{\theta}}) = g(\mathbf{h}),$$

where

- $\hat{d}$ is the predicted domain,
- $\hat{a}$ is the predicted action,
- $\hat{\mathbf{c}}$ denotes categorical slot predictions,
- $\hat{\boldsymbol{\theta}}$ denotes the predicted continuous parameter vector.

The neural network therefore performs a structured mapping

$$\mathbf{x} \longmapsto (\hat{d}, \hat{a}, \hat{\boldsymbol{\theta}}).$$

13.2 ScientificIntent Representation

The predicted quantities are assembled into a structured object,

$$\mathcal{I} = (\hat{d}, \hat{a}, \hat{\mathbf{c}}, \hat{\boldsymbol{\theta}}),$$

referred to as the **ScientificIntent**.

This representation serves as the interface between language understanding and scientific computation. It contains the information required to select the appropriate computational routine and supply its numerical parameters.

13.3 Domain Executor Selection

Based on the predicted domain and action, the framework dispatches execution to the corresponding scientific module,

$$\mathcal{E} = \Phi(\mathcal{I}),$$

where

$$\Phi : \mathcal{I} \to \text{Hydrogen}, \text{Single-Qubit}, \text{Multi-Qubit}, \ldots$$

denotes the routing operation.

Each executor encapsulates algorithms specific to its scientific domain while sharing a common interface with the natural language system.

13.4 Forward Modelling

For forward simulation tasks, the executor evaluates a deterministic mathematical model

$$\mathbf{y} = f(\hat{\boldsymbol{\theta}}),$$

where

- $\hat{\boldsymbol{\theta}}$ is the parameter vector predicted by the neural network,
- f denotes the domain-specific physical or mathematical model,
- $\mathbf{y}$ is the computed scientific output.

Examples include:

- Hydrogen emission spectra generated from transition energies, amplitudes, instrumental broadening, and background terms.
- Time evolution of quantum states under a specified Hamiltonian.
- Multi-qubit observables computed from coupled quantum systems.

The neural network predicts the parameters; the executor performs the scientific calculation.

13.5 Bayesian Inference

When parameter estimation is requested, the executor instead evaluates a statistical inference procedure.

Given observed data

$$D,$$

the executor computes or samples from the posterior distribution

$$P(\boldsymbol{\theta} \mid D) \propto P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta}),$$

using algorithms such as Markov Chain Monte Carlo (MCMC).

The resulting posterior samples may then be summarized through posterior means, credible intervals, or other statistical diagnostics.

13.6 Separation Between Learning and Scientific Computation

An important architectural principle of the framework is the separation between **parameter prediction** and **scientific evaluation**.

The transformer-based neural network approximates

$$\mathbf{x} \longmapsto \hat{\boldsymbol{\theta}},$$

mapping natural language into structured scientific parameters.

The domain executor subsequently evaluates

$$\hat{\boldsymbol{\theta}} \longmapsto f(\hat{\boldsymbol{\theta}}),$$

or performs Bayesian inference based on those parameters.

Consequently, the neural network does **not** replace the underlying physical models. Instead, it functions as an intelligent interface that translates human language into numerical inputs for scientifically grounded computational algorithms.

13.7 End-to-End Mathematical View

The complete execution pipeline may therefore be summarized as the composition

$$\boxed{\mathbf{x};\xrightarrow{;\text{Transformer};}(\hat{d},\hat{a},\hat{\boldsymbol{\theta}});\xrightarrow{;\text{ScientificIntent};}\mathcal{E};\xrightarrow{;\text{Forward Model or Bayesian Inference};}\mathbf{y}}$$

where the first stage performs learned language understanding and parameter prediction, while the final stage executes deterministic or probabilistic scientific computations using domain-specific mathematical models.

14. Assumptions and Limitations

The mathematical models and learning algorithms employed in the Scientific AI Framework rely on several assumptions and numerical approximations. Understanding these assumptions is important when interpreting model predictions and inference results.

14.1 Gaussian Noise Assumption

For regression and Bayesian inference tasks, observational errors are frequently modeled as additive Gaussian noise. Given a forward model

$$y = f(\boldsymbol{\theta}),$$

the observed data are assumed to satisfy

$$y_{\text{obs}} = f(\boldsymbol{\theta}) + \varepsilon,$$

where

$$\varepsilon \sim \mathcal{N}(0, \sigma^2).$$

Under this assumption, the likelihood function takes the form

$$P(D \mid \boldsymbol{\theta}) \propto \exp\left(-\frac{1}{2\sigma^2}\sum_i \left[y_i - f_i(\boldsymbol{\theta})\right]^2\right).$$

This assumption is appropriate for many measurement processes but may not accurately represent all experimental noise sources.

14.2 Conditional Independence Assumptions

Several probabilistic components implicitly assume conditional independence.

For example, given model parameters $\boldsymbol{\theta}$,

$$P(D \mid \boldsymbol{\theta}) = \prod_{i=1}^{N} P(d_i \mid \boldsymbol{\theta}),$$

where d_i denotes an individual observation.

Likewise, multiple supervised objectives are optimized jointly through additive loss functions, implicitly treating the individual task contributions as separable components of the total objective.

14.3 Normalization Assumptions

Continuous regression parameters are normalized prior to training to improve numerical conditioning and optimization stability.

The learning objective is computed in the normalized parameter space, whereas denormalization is applied only when reporting interpretable quantities such as mean absolute error in physical units.

Consequently, the reported evaluation metrics correspond to the original scientific variables while optimization occurs in a numerically well-scaled representation.

14.4 Numerical Approximations

Several computations rely on numerical approximations rather than closed-form analytical solutions.

Examples include:

- Discretization of continuous energy grids.
- Numerical evaluation of Gaussian line profiles.
- Time discretization in quantum evolution simulations.
- Floating-point arithmetic used throughout optimization and inference.

These approximations introduce finite numerical error that is typically negligible relative to experimental uncertainty but cannot be eliminated entirely.

14.5 Finite Sampling in Markov Chain Monte Carlo

Bayesian inference procedures approximate posterior distributions using a finite number of Monte Carlo samples.

If

$$\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \ldots, \boldsymbol{\theta}^{(M)}$$

denotes the sampled Markov chain, posterior expectations are estimated as

$$\mathbb{E}[g(\boldsymbol{\theta})] \approx \frac{1}{M} \sum_{k=1}^{M} g(\boldsymbol{\theta}^{(k)}).$$

The approximation improves as the number of effectively independent samples increases. In practice, convergence diagnostics, burn-in selection, and chain mixing influence estimation accuracy.

14.6 Dependence on Training Distribution

The neural intent detector and slot prediction components are trained using synthetic and curated examples representative of supported scientific tasks.

Performance is therefore expected to be strongest for queries resembling the training distribution. Inputs that differ substantially in terminology, structure, or scientific scope may reduce prediction accuracy despite correct downstream physical models.

14.7 Scope of Scientific Predictions

The framework predicts structured parameters and executes established computational models but does not derive new physical laws or replace domain-specific scientific theory.

The reliability of the final numerical results depends jointly on:

- the correctness of the predicted parameters,
- the validity of the selected physical model,
- the assumptions underlying Bayesian inference where applicable, and
- the quality and representativeness of the observed or synthetic data.

15. Notation Table

Symbol	Description	Typical Implementation Variable
$\boldsymbol{\theta}$	Vector of model parameters	`params`
E	Energy	`energies`
E_i	Transition energy	`transitions[i]`
A_i	Peak amplitude or transition intensity	`amplitudes[i]`
σ	Standard deviation or Gaussian broadening parameter	`sigma_instr`
B	Constant background level	`background`
$I(E)$	Simulated intensity as a function of energy	`spectrum`
D	Observed dataset	observed arrays or input data
$P(D \mid \boldsymbol{\theta})$	Likelihood function	likelihood computation
$P(\boldsymbol{\theta})$	Prior distribution	prior evaluation
$P(\boldsymbol{\theta} \mid D)$	Posterior distribution	posterior samples
ψ	Quantum state vector	`state`
$\psi(t)$	Time-evolved quantum state	propagated state
H	Hamiltonian operator	`hamiltonian`
t	Time variable	`times`
$U(t)$	Time-evolution operator	unitary evolution
ρ	Density matrix	`density_matrix`
$\mathcal{O}$	Observable operator	`observable`
$\langle \mathcal{O} \rangle$	Expectation value of an observable	measured expectation

Symbol	Description	Typical Implementation Variable
λ	Eigenvalue or weighting coefficient (context dependent)	model-specific parameter
L	Loss function	`loss`
L_{reg}	Masked regression loss	`loss_reg`
L_{total}	Combined multi-task objective	total training loss
$\hat{\boldsymbol{\theta}}$	Predicted parameter vector	`regression_output`
$\mathbf{m}$	Regression mask indicating active parameters	`reg_mask`
$\hat{d}$	Predicted scientific domain	`domain_logits`
$\hat{a}$	Predicted action	`action_logits`
$\mathbf{x}$	Tokenized user query	`input_ids`
$\mathbf{h}$	Transformer latent representation	encoder hidden representation
$\mathcal{I}$	Structured `ScientificIntent` object	`ScientificIntent`
$f(\boldsymbol{\theta})$	Domain-specific forward model	executor forward computation
$\mathbb{E}[\cdot]$	Statistical expectation	posterior summary statistics
MAE	Mean Absolute Error	evaluation metric
MSE	Mean Squared Error	regression loss computation

The notation is used consistently throughout this guide to bridge the mathematical formulation and its corresponding implementation within the Scientific AI Framework.

16. Summary of Mathematical Models

The Scientific AI Framework integrates techniques from machine learning, Bayesian statistics, quantum mechanics, spectroscopy, and numerical optimization into a unified architecture for scientific computation. Rather than relying solely on neural networks or solely on analytical models, the framework combines learned language understanding with domain-specific mathematical formulations.

At the highest level, the framework performs the mapping

$$\boxed{\text{Natural Language} \longrightarrow \text{Structured Scientific Intent} \longrightarrow \text{Mathematical Model} \longrightarrow \text{Numerical Results}}$$

where the first stage is data-driven and the latter stages are governed by established scientific principles.

16.1 Bayesian Inference

Unknown model parameters are estimated through Bayes' theorem,

$$P(\boldsymbol{\theta} \mid D) = \frac{P(D \mid \boldsymbol{\theta}) P(\boldsymbol{\theta})}{P(D)},$$

combining prior information with observed data to obtain posterior probability distributions.

When analytical solutions are unavailable, posterior distributions are approximated using Markov Chain Monte Carlo (MCMC) sampling.

16.2 Forward Modelling

Scientific predictions are generated through deterministic forward models of the form

$$\mathbf{y} = f(\boldsymbol{\theta}),$$

where the parameter vector $\boldsymbol{\theta}$ specifies the physical system and f represents the corresponding mathematical model.

Examples implemented in the framework include hydrogen spectral synthesis and quantum dynamical simulations.

16.3 Hydrogen Spectrum Model

Hydrogen emission spectra are represented as superpositions of Gaussian line profiles,

$$I(E) = B + \sum_i A_i \exp\left(-\frac{(E - E_i)^2}{2\sigma^2}\right),$$

where transition energies, amplitudes, instrumental broadening, and background determine the resulting spectrum.

The same mathematical model is used both for synthetic data generation and Bayesian parameter inference.

16.4 Quantum Mechanical Models

Single-qubit and multi-qubit systems evolve according to the Schrödinger equation,

$$i\hbar\frac{\partial}{\partial t}\psi(t) = H\psi(t),$$

whose formal solution is

$$\psi(t) = e^{-iHt/\hbar}\psi(0).$$

Observable quantities are obtained from expectation values or measurement probabilities derived from the evolved quantum state.

16.5 Neural Intent Detection and Slot Prediction

Transformer-based language models map user queries into structured predictions consisting of

- scientific domain,
- requested action,
- categorical slots, and
- continuous regression parameters.

These predictions define a `ScientificIntent` object that serves as the interface between natural language understanding and scientific execution.

Regression parameters are represented using a global slot vector with masked optimization, allowing a single model to support heterogeneous scientific domains while ignoring parameters irrelevant to a particular task.

16.6 Multi-Task Learning

Training jointly optimizes classification and regression objectives through a combined loss function,

$$L_{\text{total}} = L_{\text{domain}} + L_{\text{action}} + \sum_k L_{\text{cat}}^{(k)} + \lambda_{\text{reg}} L_{\text{reg}}.$$

Continuous targets are normalized during optimization to improve numerical stability, while denormalization is reserved for reporting interpretable evaluation metrics.

16.7 Execution Architecture

The neural network predicts structured scientific parameters but does not itself perform scientific calculations.

Instead, execution proceeds conceptually as

$$\boxed{\mathbf{x}; \xrightarrow{;\text{Neural Intent Detector};} \mathcal{I}; \xrightarrow{;\text{Domain Executor};} f(\boldsymbol{\theta}); \xrightarrow{;} \text{Scientific Output}}$$

where:

- $\mathbf{x}$ denotes the user query,
- $\mathcal{I}$ is the predicted `ScientificIntent`,
- the domain executor evaluates the appropriate mathematical model, and
- the final output consists of physically meaningful numerical results or Bayesian posterior estimates.

16.8 Overall Perspective

The central philosophy of the Scientific AI Framework is that machine learning should **facilitate scientific computation rather than replace it**.

Neural networks provide robust interpretation of natural language and estimation of structured parameters, while established mathematical models continue to govern physical simulation, probabilistic inference, and numerical prediction. This separation preserves scientific interpretability while enabling an intuitive natural-language interface for complex computational workflows.